



## Capítulo 3 - Organização do programa em módulos

Separar as tarefas do programa ajuda a compreender, testar e ampliar a automação ferroviária.

|                      |                                                                                                                                                                                |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Introdução           | Nos capítulos anteriores, criamos funções e utilizamos decisões e repetições. Agora vamos distribuir essas funções em partes organizadas, cada uma responsável por uma tarefa. |
| Objetivo             | Organizar um programa em módulos e compreender como separar declarações e implementações em arquivos.                                                                          |
| Meta de aprendizagem | Ao final, você deverá conseguir criar um programa com um arquivo principal, um arquivo de cabeçalho e um arquivo de implementação, mantendo clara a comunicação entre eles.    |

### 1. O que significa organizar em módulos?

Neste curso, um módulo é uma parte do programa dedicada a uma responsabilidade. Podemos ter, por exemplo, uma parte para ler entradas, outra para controlar desvios e outra para apresentar mensagens.

Esses módulos podem começar como grupos de funções no mesmo arquivo. Quando o programa crescer, podemos separá-los em arquivos próprios.

Assim como organizamos os componentes físicos da ferrovia por função, podemos organizar o código. Um módulo de desvios reúne as funções relacionadas aos desvios; um módulo de mensagens reúne as funções de apresentação de informações.

Aqui, “módulo” significa uma divisão prática do nosso código. Não utilizaremos o recurso específico de módulos da linguagem C++ chamado `import`.

### 2. Por que separar as responsabilidades?

Quando todas as instruções ficam misturadas, uma pequena alteração pode ser difícil de localizar. A divisão em módulos facilita encontrar o trecho responsável por cada comportamento.

- Melhora a leitura do programa.
- Permite testar uma tarefa por vez.
- Reduz a repetição de instruções.
- Facilita acrescentar novos elementos à automação.



A divisão, por si só, não torna o programa mais rápido nem faz as tarefas acontecerem simultaneamente. As funções continuam sendo executadas conforme as chamadas e a sequência das instruções.

### 3. O papel do programa principal

O arquivo principal deve mostrar a sequência geral do programa. Os detalhes de cada tarefa ficam nas funções correspondentes.

```
void setup() {  
    startMessages();  
    showStartupMessage();  
}  
  
void loop() {  
    // Future tasks will be called here.  
}
```

Ao ler esse trecho, conseguimos perceber que o programa inicia a comunicação de mensagens e apresenta uma mensagem inicial. Não precisamos conhecer todos os detalhes internos das funções para compreender a sequência.

Na automação completa, o loop() poderá chamar funções de leitura, decisão e comando. Cada chamada deve executar uma tarefa curta para permitir que o programa continue acompanhando as entradas.

### 4. Os três arquivos do exercício

Vamos criar um módulo chamado Messages, dedicado às mensagens do Monitor Serial. O programa terá três arquivos na mesma pasta:

| Arquivo                     | Responsabilidade                                                          |
|-----------------------------|---------------------------------------------------------------------------|
| intermediate_03_modules.ino | Arquivo principal: contém setup(), loop() e as chamadas das funções.      |
| Messages.h                  | Cabeçalho: declara as funções disponíveis para outras partes do programa. |
| Messages.cpp                | Implementação: contém as instruções que cada função executa.              |

A extensão .h identifica o cabeçalho. A extensão .cpp identifica um arquivo de código C++. O arquivo .ino é o programa principal do projeto na Arduino IDE.

O nome da pasta do projeto deve ser intermediate\_03\_modules, igual ao nome do arquivo principal sem a extensão. Os três arquivos precisam permanecer juntos nessa pasta.



### 5. Declaração e implementação

A declaração informa o nome da função, os parâmetros recebidos e o tipo de retorno. Ela termina com ponto e vírgula e não contém as instruções da tarefa.

```
void showTurnoutCommand(int turnoutNumber, int angle);
```

A implementação apresenta a função completa, com as instruções entre chaves:

```
void showTurnoutCommand(int turnoutNumber, int angle) {  
    Serial.print("Turnout D");  
    Serial.print(turnoutNumber);  
    Serial.print(" - Command angle: ");  
    Serial.println(angle);  
}
```

A declaração e a implementação precisam corresponder: mesmo nome, mesmos tipos de parâmetros, na mesma ordem, e mesmo tipo de retorno.

O arquivo principal consulta as declarações do cabeçalho. Durante a construção do programa, o código das funções no arquivo .cpp é reunido ao código principal.

### 6. Criando o cabeçalho Messages.h

Este é o conteúdo completo do arquivo Messages.h:

```
#ifndef MESSAGES_H  
#define MESSAGES_H  
  
void startMessages();  
void showStartupMessage();  
void showTrackStatus(bool trackOccupied);  
void showTurnoutCommand(int turnoutNumber, int angle);  
void showEndMessage();  
  
#endif
```

As linhas `#ifndef`, `#define` e `#endif` formam uma proteção para que o conteúdo do cabeçalho não seja incluído repetidamente na mesma unidade de compilação.

Por enquanto, basta manter essa estrutura como apresentada e colocar as declarações dentro dela. Essas linhas não são as condições `if` executadas pelo programa: são instruções utilizadas durante o preparo do código para compilação.



## 7. Criando a implementação Messages.cpp

Este é o conteúdo completo do arquivo Messages.cpp:

```
#include <Arduino.h>
#include "Messages.h"

void startMessages() {
    Serial.begin(115200);
    delay(1000);
}

void showStartupMessage() {
    Serial.println("EFMN 74 - Program organized in modules");
}

void showTrackStatus(bool trackOccupied) {
    if (trackOccupied) {
        Serial.println("Track occupied");
    } else {
        Serial.println("Track clear");
    }
}

void showTurnoutCommand(int turnoutNumber, int angle) {
    Serial.print("Turnout D");
    Serial.print(turnoutNumber);
    Serial.print(" - Command angle: ");
    Serial.println(angle);
}

void showEndMessage() {
    Serial.println("Exercise completed");
}
```

#include <Arduino.h> disponibiliza os recursos do ambiente Arduino, como Serial e delay(), neste arquivo C++.

#include "Messages.h" inclui o cabeçalho do nosso módulo. As aspas indicam a inclusão de um arquivo do projeto.

Nos arquivos .ino, a Arduino IDE prepara automaticamente algumas declarações. Em arquivos .cpp, escrevemos as inclusões e declarações necessárias. Por isso, utilizamos Arduino.h e o cabeçalho do próprio módulo.

A pausa de um segundo em startMessages() será executada somente quando essa função for chamada. Neste exercício, a chamada ocorre uma vez, em setup().



## 8. Criando o arquivo principal

Este é o conteúdo completo de `intermediate_03_modules.ino`:

```
#include "Messages.h"

void setup() {
  startMessages();
  showStartupMessage();

  showTrackStatus(false);
  showTurnoutCommand(6, 67);
  showTurnoutCommand(6, 97);

  showEndMessage();
}

void loop() {
}
```

O arquivo principal chama as funções declaradas em `Messages.h`. As instruções de cada função estão em `Messages.cpp`.

Não precisamos escrever `#include "Messages.cpp"`. A Arduino IDE compila o arquivo `.cpp` da pasta do projeto e reúne o resultado ao programa.

O exercício apenas apresenta mensagens. Ele não lê um sensor real e não movimenta o D6. O valor `false` simula um trecho livre; os ângulos 67 e 97 são apenas informações mostradas na tela.

## Exercício prático

Meta: montar um projeto com três arquivos e conferir o funcionamento do módulo de mensagens.

1. Abra a Arduino IDE e crie um novo programa.
2. Salve o projeto com o nome `intermediate_03_modules`.
3. No arquivo principal, cole o código do item 8.
4. No menu de abas do editor, utilize a opção de criar uma nova aba e informe `Messages.h`.
5. Cole nessa aba o código do item 6.
6. Crie outra aba chamada `Messages.cpp` e cole o código do item 7.
7. Salve todos os arquivos e confira os nomes, incluindo as extensões.
8. Conecte uma ESP32 de bancada ao computador por USB e selecione a placa e a porta corretas.
9. Utilize Verificar/Compilar. Corrija eventuais erros antes de enviar.
10. Envie o programa, abra o Monitor Serial em 115200 baud e pressione EN ou RESET.

Resultado esperado:



EFMN 74 - Program organized in modules  
Track clear  
Turnout D6 - Command angle: 67  
Turnout D6 - Command angle: 97  
Exercise completed

## Alterações para praticar

1. No arquivo principal, troque `showTrackStatus(false)` por `showTrackStatus(true)`. Confira a mensagem.
2. No arquivo principal, troque o número do desvio de 6 para 9 nas duas chamadas. Não altere o módulo.
3. Em `Messages.cpp`, altere a mensagem inicial. Compile e envie novamente para conferir a mudança.
4. Acrescente a declaração `void showTestMessage();` em `Messages.h`, antes de `#endif`.
5. Em `Messages.cpp`, implemente essa função para escrever `Module test completed`.
6. Chame `showTestMessage();` em `setup()`, antes de `showEndMessage();`, e confira o resultado.

Faça uma alteração de cada vez. No último exercício, observe que acrescentar uma função envolve declarar, implementar e chamar.

## 9. Como ampliar a organização

Quando o programa crescer, podemos criar outros módulos. A divisão abaixo é uma proposta de organização para os próximos capítulos, e não uma configuração já instalada na ferrovia:

| Módulo   | Responsabilidade prevista                             |
|----------|-------------------------------------------------------|
| Messages | Apresentar informações e avisos.                      |
| Turnouts | Reunir as funções de comando dos desvios.             |
| Sensors  | Reunir as funções de leitura dos sensores.            |
| Routes   | Reunir as funções de seleção e verificação das rotas. |

Comece com poucos módulos, cada um com uma responsabilidade definida. Evite criar muitos arquivos antes de haver necessidade.

Para trocar informações entre módulos, prefira funções com parâmetros e retorno bem definidos. Não coloque definições de variáveis globais comuns em vários arquivos: isso pode gerar duplicações e dificultar a manutenção.



## 10. Erros comuns

| Erro                                                                       | Como corrigir                                                                                             |
|----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Aparece "Messages.h: No such file or directory".                           | Confira o nome, a extensão e se o arquivo está na pasta do projeto.                                       |
| Aparece uma mensagem de função não declarada.                              | Confira se a declaração está no cabeçalho e se ele foi incluído no arquivo que chama a função.            |
| Aparece "undefined reference" para uma função.                             | Confira se há uma implementação correspondente no arquivo .cpp e se ele pertence ao projeto.              |
| Aparece uma mensagem de definição duplicada.                               | Confira se a mesma função foi implementada em mais de um arquivo ou se o .cpp foi incluído indevidamente. |
| Usar messages.h em um lugar e Messages.h em outro.                         | Utilize a mesma escrita em todos os arquivos, respeitando maiúsculas e minúsculas.                        |
| A declaração recebe dois parâmetros, mas a implementação recebe apenas um. | Faça a declaração e a implementação corresponderem.                                                       |

## Resumo do capítulo

- Um módulo reúne funções relacionadas a uma responsabilidade.
- O arquivo principal apresenta a sequência geral do programa.
- O cabeçalho .h declara as funções disponíveis.
- O arquivo .cpp implementa essas funções.
- #include permite utilizar as declarações do cabeçalho.
- Os arquivos do exercício devem permanecer na mesma pasta do projeto.
- Separar o código facilita a leitura e os testes, mas não cria execução simultânea.

Próximo capítulo: Controle de vários desvios.